

IS4103

Operating Systems

Interprocess Communication

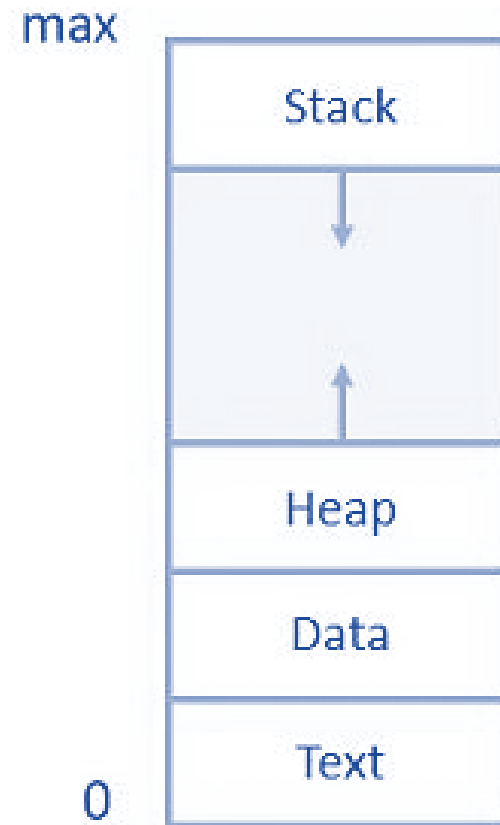
T.Kayalvizhy

Learning Objectives

Learning Outcome	Description
ILO2	Explain how memory, I/O devices, files, processes & threads are managed

What is a Process?

- Program in execution
- Has its own address space
- Has stack, heap, registers
- Executes independently



Why Do Processes Need to Communicate?

- Information sharing
- Computation speed-up
- Modularity
- Convenience
- Example:
 - In a browser like Google Chrome:
 - One process handles UI
 - One handles rendering
 - One handles networking

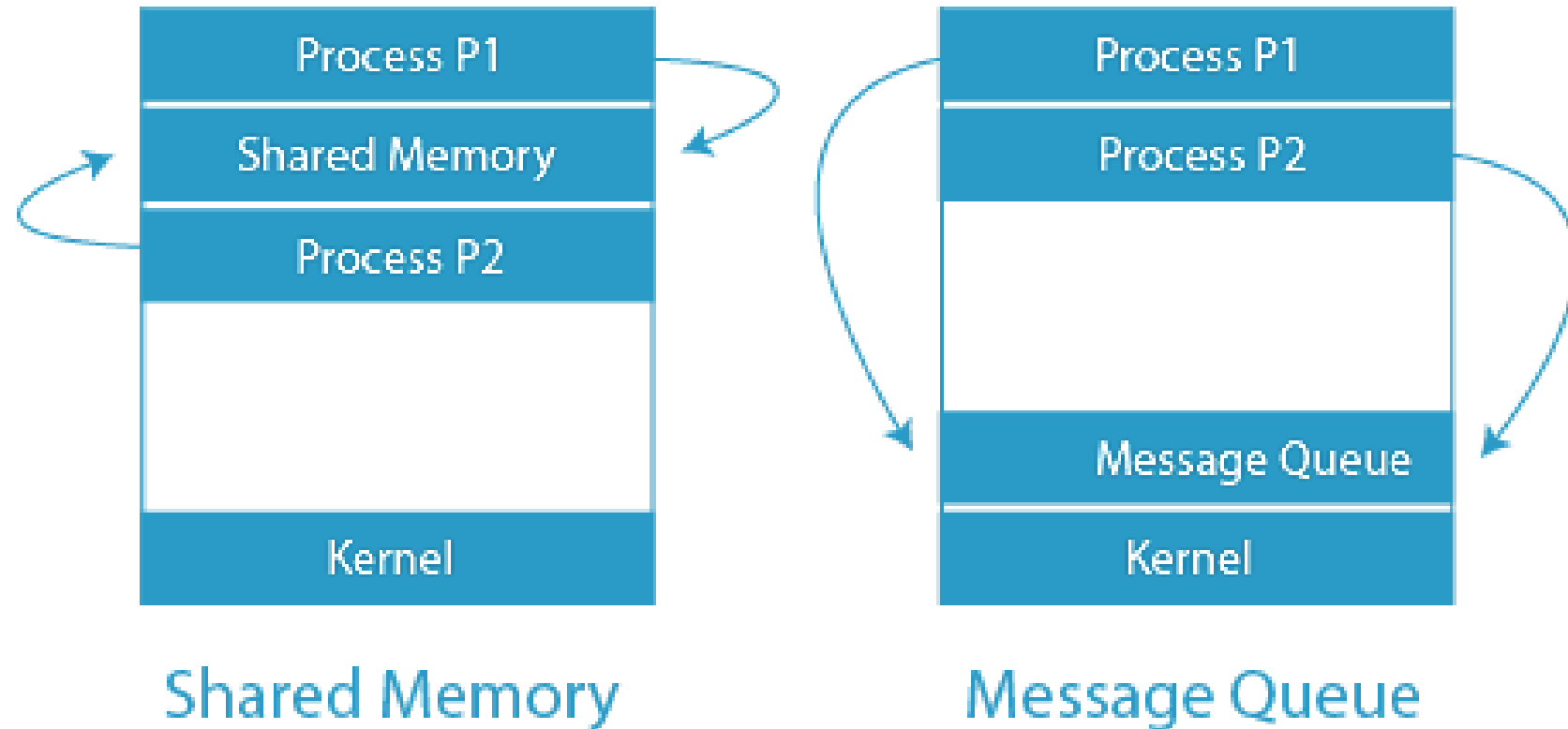
Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is an operating system mechanism enabling independent processes to exchange data, synchronize actions, and share information.

It ensures:

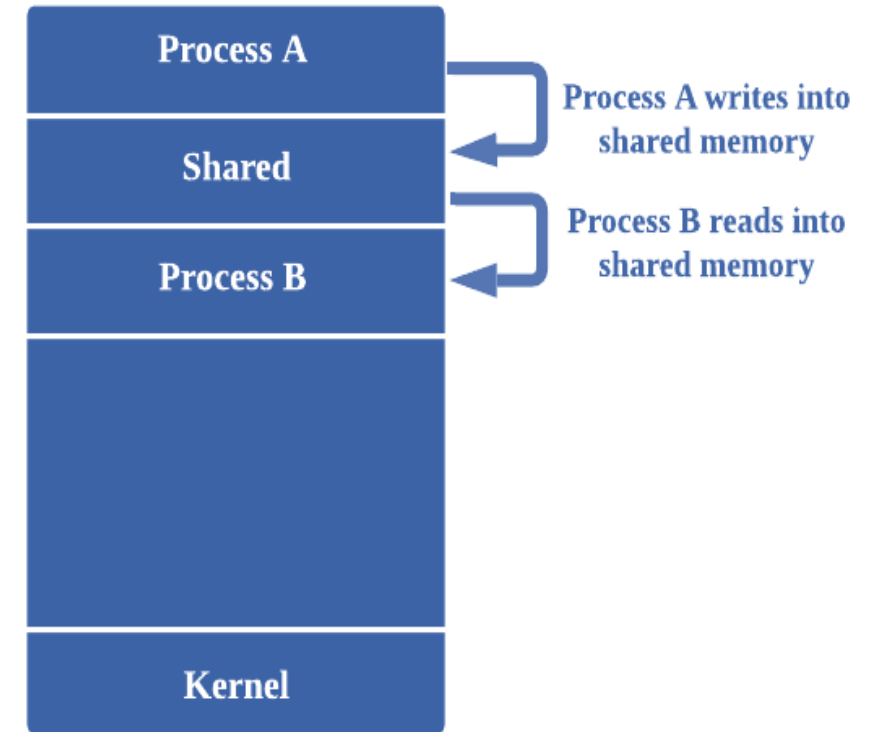
- Safe communication
- Data integrity
- Controlled access

IPC Approaches



Shared Memory Model

- OS creates a shared segment
- Processes attach
- Read/write like normal memory



After setup, the kernel is not involved in every read/write.

Shared Memory Advantages & Issues

Advantages:

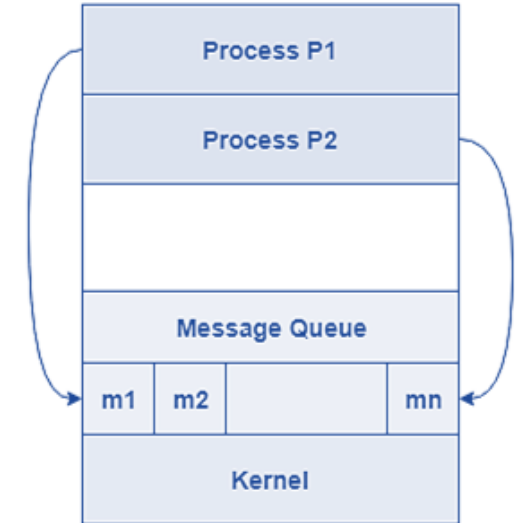
- Very fast
- No message copying
- Efficient for large data

Problems:

- No automatic synchronization
- Race conditions possible

Message Queue

- Processes communicate by exchanging messages
- No shared address space
- OS kernel manages communication
- Basic operations:
 - send(message)
 - receive(message)
- Communication Types:
 - Direct or Indirect
 - Blocking (Synchronous) or Non-blocking (Asynchronous)

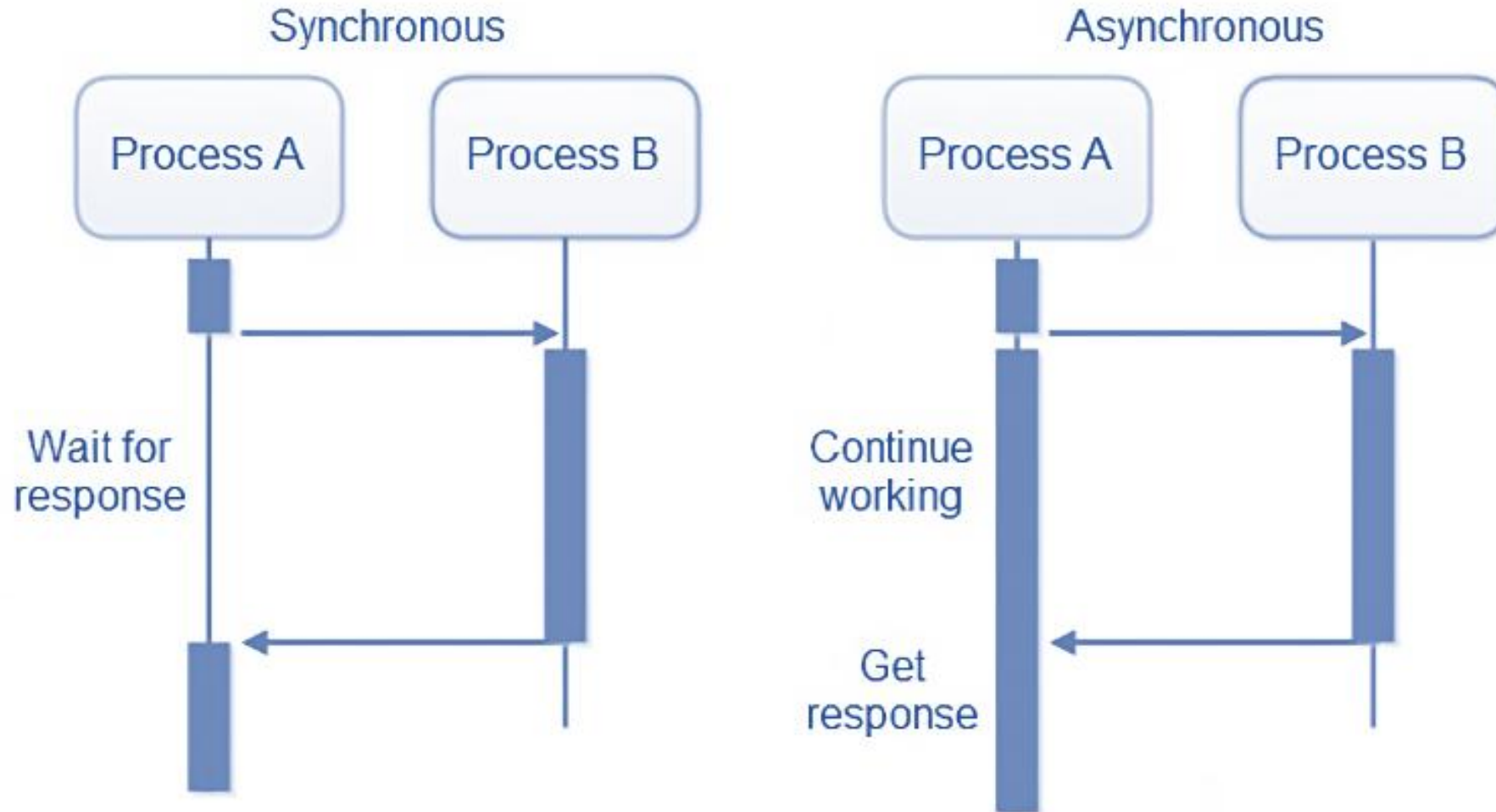


Message Passing Model

Direct vs Indirect Communication

- Direct:
 - Process A sends directly to Process B.
 - Both must know each other.
- Indirect:
 - Uses mailbox/port.
 - Processes are sent to the mailbox instead of being processed directly.

Blocking vs Non-Blocking Communication



Blocking vs Non-Blocking Communication

Aspect	Blocking	Non-Blocking
Execution Flow	Halts program execution until completion.	<u>Allows</u> program to continue with other tasks.
Efficiency	May waste CPU time waiting for a task.	Utilizes CPU efficiently for other tasks.
Complexity	Simpler to implement and debug.	Requires additional <u>constructs</u> like callbacks.
Responsiveness	Can make programs unresponsive.	Maintains responsiveness of applications.

Thank you

Any Questions?